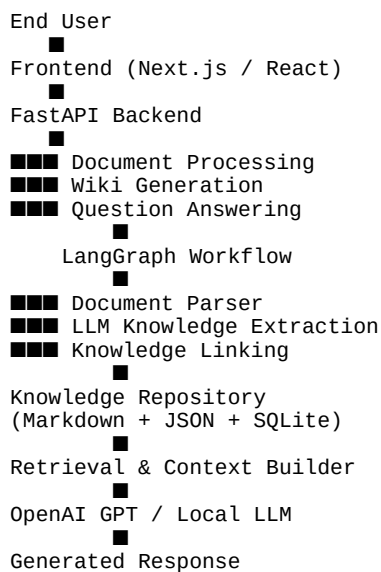


WikiLLM-Based Knowledge Management System

System Overview

This document summarizes the proposed architecture, components, workflow, storage, deployment, and technology stack for the WikiLLM-based Knowledge Management & Question Answering System.

High-Level Architecture



Frontend Layer

Responsibilities: User Authentication, Document Upload, Browse Wiki Pages, Ask Questions, View Responses, Processing Status.

Technology: Next.js, React, Tailwind CSS, TypeScript.

Backend Layer

FastAPI backend handling REST APIs, authentication, file management, business logic, AI orchestration, logging and error handling.

Document Processing

Supports PDF, DOCX and TXT. Workflow: Upload → Extract Text → Clean Formatting → Pass to LLM.

Libraries: PyMuPDF, python-docx.

Knowledge Extraction

LLM extracts entities, concepts, facts, relationships and summaries into structured JSON.

Wiki Generation

Automatically creates interconnected Markdown wiki pages with cross-links.

Knowledge Repository

Stores Markdown wiki pages, JSON relationships and SQLite metadata.

Knowledge Linking Engine

Builds relationships between pages to form a navigable knowledge graph.

Question Answering

Retrieves relevant wiki pages, builds context and generates answers using the LLM.

LangGraph Workflow

Upload → Parse → Extract Knowledge → Generate Wiki → Store → Retrieve → Answer.

Database Design

SQLite: users, documents, metadata, processing status.

Markdown: wiki pages.

JSON: relationships and knowledge graph.

Backend Structure

app/api, services, models, database, workflows (LangGraph), main.py

Frontend Structure

src/pages, components, services, hooks, layouts, types, styles

API Endpoints

POST /upload

GET /documents

GET /wiki/{id}

POST /chat

GET /status/{id}

DELETE /documents/{id}

Deployment

Browser → Next.js → FastAPI → LangGraph → OpenAI/Ollama → Knowledge Repository → SQLite

Technology Stack

Frontend: Next.js, React, Tailwind CSS

Backend: FastAPI

Workflow: LangGraph

LLM: OpenAI GPT or Ollama
Storage: Markdown, JSON, SQLite
Optional Vector Store: ChromaDB / FAISS
Deployment: Docker, Nginx, GitHub

Conclusion

The proposed architecture extends beyond traditional RAG by converting uploaded documents into a persistent, structured Wiki-based knowledge repository. The system is modular, scalable, and supports future enhancements such as hybrid retrieval, contradiction detection, and knowledge graph visualization.